

Gemini 3.8 Flash is now available. [Try it out](#)

(https://aistudio.google.com/prompts/new_chat?model=gemini-3.8-flash).

Prompt design strategies

Prompt design is the process of creating prompts, or natural language requests, that elicit accurate, high quality responses from a language model.

This page introduces basic concepts, strategies, and best practices to get you started designing prompts to get the most out of Gemini AI models.

Topic-specific prompt guides

Looking for more specific prompt strategies? Check out our other prompting guides on:

- [Prompting with media files](/gemini-api/docs/files#prompt-guide) (/gemini-api/docs/files#prompt-guide)
- [Prompting for image generation](/gemini-api/docs/image-generation#prompt-guide) (/gemini-api/docs/image-generation#prompt-guide)
- [Prompting for video generation](/gemini-api/docs/video#prompt-guide) (/gemini-api/docs/video#prompt-guide)

You can find other sample prompts in the [prompt gallery](/gemini-api/prompts) (/gemini-api/prompts) meant to interactively showcase many of the concepts shared in this guide.

Clear and specific instructions

An effective and efficient way to customize model behavior is to provide it with clear and specific instructions. Instructions can be in the form of a question, step-by-step tasks, or as complex as mapping out a user's experience and mindset.

Input

Input is the required text in the prompt that you want the model to provide a response to. Inputs can be a question that the model answers (question input), a task the model performs (task input), an entity the model operates on (entity input), or partial input that the model completes or continues (completion input).

Input type	Prompt	Generated output
Question	What's a good name for a flower shop that specializes in selling bouquets of dried flowers? Create a list of 5 options with just the names.	Here are 5 names for a flower shop specializing in dried flowers: 1. Everlasting Blooms 2. Dried & Delightful 3. The Petal Preserve 4. Whispers of Wildflowers 5. Forever Blooms
Task	Give me a simple list of just the things that I must bring on a camping trip. The list should have 5 items.	1. Tent: Your shelter from the elements. 2. Sleeping Bag: Stay warm and comfortable. 3. Sleeping Pad: Insulation and cushioning for a good night's sleep. 4. Headlamp or Flashlight: Light for nighttime activities. 5. First Aid Kit: Be prepared for minor injuries.
Entity	Classify the following items as [large, small]: Elephant Mouse Snail	Here's the classification: * Elephant: Large * Mouse: Small * Snail: Small

Partial input completion

Generative language models work like an advanced auto completion tool. When you provide partial content, the model can provide the rest of the content or what it thinks is a continuation of that content as a response. When doing so, if you include any examples or context, the model can take those examples or context into account.

The following example provides a prompt with an instruction and an entity input:

:

the given order, return a JSON object that has the fields cheeseburger, hamburger, fries, or drink, with the value being the quantity.

Example: A burger and a drink.

se:

```
{
  "cheeseburger": 0,
  "hamburger": 1,
  "fries": 0,
  "drink": 1
}
```

(gemini-2.

While the model did as prompted, writing out the instructions in natural language can sometimes be challenging and it leaves a lot to the model's interpretation. For example, a restaurant's menu might contain many items. To reduce the size of the JSON response, you probably want to omit the items that weren't ordered. In this case, you can give an example and a response prefix and let the model complete it:

:

The fields are cheeseburger, hamburger, fries, and drink.

Example: Give me a cheeseburger and fries

it:

```
{
  "cheeseburger": 1,
  "fries": 1
}
```

Example: I want two burgers, a drink, and fries.

it:

se:

```
hamburger": 2,  
"hamburger": 1,  
"hamburger": 1
```

(gemini-2.

Notice how "cheeseburger" was excluded from the output because it wasn't a part of the order.

While you can specify the format of simple JSON response objects using prompts, we recommend using Gemini API's [structured output](https://cloud.google.com/ai/gemini-api/docs/structured-output) (/gemini-api/docs/structured-output) feature when specifying a more complex JSON Schema for the response.

Constraints

Specify any constraints on reading the prompt or generating a response. You can tell the model what to do and not to do. For example, you can specify a constraint in the prompt on how long you want a summary to be:

:

Summarize this text in one sentence:

A quantum computer exploits quantum mechanical phenomena to perform calculations exponentially faster than any modern traditional computer. At very tiny scales, physical matter acts as particles and as waves, and quantum computing uses specialized hardware to leverage this behavior. Understanding the operating principles of quantum devices is beyond the scope of classical physics. When deployed at scale, quantum computers could be used in a wide variety of applications such as: in cybersecurity to break existing encryption methods while helping researchers create new quantum computing technology to develop better weather forecasting etc. However, the current state of quantum computing is still largely experimental and impractical.

se:

Exploiting quantum mechanical phenomena, quantum computers can perform calculations potentially faster than traditional computers for potential applications like cryptography and meteorology, although they are currently largely experimental and theoretical.

(gemini-2.

Response format

You can give instructions that specify the format of the response. For example, you can ask for the response to be formatted as a table, bulleted list, elevator pitch, keywords, sentence, or paragraph. The following system instruction tells the model to be more conversational in its response:

instruction

All questions should be answered comprehensively with details, unless the user requests a concise response specifically.

What is a smart way to make a business that sells DVD's in 2026?

se:

Running a business selling DVDs in 2026 presents significant challenges due to the overwhelming dominance of digital streaming and on-demand services; a "smart" approach would therefore require a highly specialized niche, a strong value proposition beyond just the disc, and likely a focus on community or collector markets rather than mainstream consumption.

Here's a breakdown of what a smart DVD business in 2026 would likely entail:

Extreme Niche Specialization: You cannot compete with streaming libraries. Your business must focus on what digital doesn't offer easily or at all. This could include:

: Out-of-Print & Rare Titles: Sourcing films or TV shows that have never been released or are currently unavailable on major streaming platforms.
...

(gemini-2.

Format responses with the completion strategy

The completion strategy (#completion) can also help format the response. The following example prompts the model to create an essay outline:

:

Generate an outline for an essay about hummingbirds.

se:

Here is a possible outline for an essay about hummingbirds:

Introduction

Hook: Start with a captivating fact or description about hummingbirds (e.g., their rapid flight, iridescent colors, or tiny size).

Background: Briefly introduce hummingbirds – where they are found (Americas), their classification as unique birds.

Thesis Statement: State the main point of the essay, which could be about the remarkable adaptations, behaviors, and ecological importance of hummingbirds.

Appearance and Identification

(gemini-2.

The prompt didn't specify the format for the outline and the model chose a format for you. To get the model to return an outline in a specific format, you can add text that represents the start of the outline and let the model complete it based on the pattern that you initiated.

:

Write an outline for an essay about hummingbirds.
Introduction

Here is:

Here is an outline for an essay about hummingbirds, starting from the introduction provided:

Introduction

Hook: Start with a captivating fact or description about hummingbirds (e.g., their iridescent colors, or tiny size).

Background: Briefly introduce hummingbirds – where they are found (Americas), their classification as unique birds, and their general characteristics.

Thesis Statement: State the main point of the essay, focusing on the remarkable adaptations, behaviors, and ecological significance that make hummingbirds extraordinary.

(gemini-2.0)

Zero-shot vs few-shot prompts

You can include examples in the prompt that show the model what getting it right looks like. The model attempts to identify patterns and relationships from the examples and applies them when generating a response. Prompts that contain a few examples are called *few-shot* prompts, while prompts that provide no examples are called *zero-shot* prompts. Few-shot prompts are often used to regulate the formatting, phrasing, scoping, or general patterning of model responses. Use specific and varied examples to help the model narrow its focus and generate more accurate results.

We recommend to always include few-shot examples in your prompts. Prompts without few-shot examples are likely to be less effective. In fact, you can remove instructions from your prompt if your examples are clear enough in showing the task at hand.

The following zero-shot prompt asks the model to choose the best explanation.

:

we choose the best explanation to the question:

Question: How is snow formed?

Explanation1: Snow is formed when water vapor in the air freezes into ice crystals in the atmosphere, which can combine and grow into snowflakes as they fall through the atmosphere and accumulate on the ground.

Explanation2: Water vapor freezes into ice crystals forming snow.

Answer:

Answer:

Explanation1 is the better explanation because it provides more detail on the process, including how ice crystals combine and grow into snowflakes as they fall through the atmosphere.

(gemini-2.0)

If your use case requires the model to produce concise responses, you can include examples in the prompt that give preference to concise responses.

The following prompt provides two examples that show preference to the shorter explanations. In the response, you can see that the examples guided the model to choose the shorter explanation (**Explanation2**) as opposed to the longer explanation (**Explanation1**) like it did previously.

:

Here are some examples showing a question, explanation, and answer format:

Question: Why is the sky blue?

Explanation1: The sky appears blue because of Rayleigh scattering, which causes shorter blue wavelengths of light to be scattered more easily than longer red wavelengths, making the sky look blue.

Explanation2: Due to Rayleigh scattering effect.

Answer: Explanation2

Question: What is the cause of earthquakes?

Explanation1: Sudden release of energy in the Earth's crust.

Explanation2: Earthquakes happen when tectonic plates suddenly slip or break apart, causing a release of energy that creates seismic waves that can shake the ground and cause damage.

Answer: Explanation1

Answer the following question given the example formats above:

Question: How is snow formed?

Explanation1: Snow is formed when water vapor in the air freezes into ice crystals in the atmosphere, which can combine and grow into snowflakes as they fall through the atmosphere and accumulate on the ground.

Explanation2: Water vapor freezes into ice crystals forming snow.

Answer:

Answer:

Answer: Explanation2

(gemini-2.0)

Optimal number of examples

Models like Gemini can often pick up on patterns using a few examples, though you may need to experiment with the number of examples to provide in the prompt for the best results. At the same time, if you include too many examples, the model may start to overfit

(<https://developers.google.com/machine-learning/glossary#overfitting>) the response to the examples.

Consistent formatting

Make sure that the structure and formatting of few-shot examples are the same to avoid responses with undesired formats. One of the primary objectives of adding few-shot examples in prompts is to show the model the response format. Therefore, it is essential to ensure a consistent format across all examples, especially paying attention to XML tags, white spaces, newlines, and example splitters.

Add context

You can include instructions and information in a prompt that the model needs to solve a problem, instead of assuming that the model has all of the required information. This contextual information helps the model understand the constraints and details of what you're asking for it to do.

The following example asks the model to give troubleshooting guidance for a router:

:

What should I do to fix my disconnected wifi? The light on my Google router is yellow and blinking slowly.

se:

A yellow blinking light on your Google Wifi router usually indicates a problem with the internet connection. Here are some steps to troubleshoot and fix the issue:

Restart your router and modem:

- Unplug your modem from the power outlet and wait for 30 seconds.
- Plug your modem back in and wait for it to fully boot up.
- Unplug your Google Wifi router and wait for 30 seconds.
- Plug your Google Wifi router back in and wait for it to fully boot up.

(gemini-2.

The response looks like generic troubleshooting information that's not specific to the router or the status of the LED indicator lights.

To customize the response for the specific router, you can add to the prompt the router's troubleshooting guide as context for it to refer to when providing a response.

:

For the question using the text below. Respond with only the text provided.
Question: What should I do to fix my disconnected wifi? The light on my Google Wifi router is yellow and blinking slowly.

Answer: Slowly pulsing yellow

It means: There is a network error.

What to do:

Make sure that the Ethernet cable is connected to both your router and your modem and both devices are turned on. You might need to unplug and plug in each device again.

Answer: Fast blinking yellow

It means: You are holding down the reset button and are factory resetting this device.
What to do:

ou keep holding down the reset button, after about 12 seconds, the light will turn
l yellow. Once it is solid yellow, let go of the factory reset button.

:: Solid yellow

it means: Router is factory resetting.

to do:

can take up to 10 minutes. When it's done, the device will reset itself and start
ng white, letting you know it's ready for setup.

:: Solid red

it means: Something is wrong.

to do:

.cal failure. Factory reset the router. If the light stays red, contact Wifi custom
ort.

se:

: that the Ethernet cable is connected to both your router and your modem and both
es are turned on. You might need to unplug and plug in each device again.

(gemini-2.

Break down prompts into components

For use cases that require complex prompts, you can help the model manage this complexity by breaking things down into simpler components.

1. **Break down instructions:** Instead of having many instructions in one prompt, create one prompt per instruction. You can choose which prompt to process based on the user's input.
2. **Chain prompts:** For complex tasks that involve multiple sequential steps, make each step a prompt and chain the prompts together in a sequence. In this sequential chain of prompts, the output of one prompt in the sequence becomes the input of the next prompt. The output of the last prompt in the sequence is the final output.
3. **Aggregate responses:** Aggregation is when you want to perform different parallel tasks on different portions of the data and aggregate the results to produce the final output. For example, you can tell the model to perform one operation on the first part of the data, perform another operation on the rest of the data and aggregate the results.

Experiment with model parameters

Each call that you send to a model includes parameter values that control how the model generates a response. The model can generate different results for different parameter values. Experiment with different parameter values to get the best values for the task. The parameters available for different models may differ. The most common parameters are the following:

1. **Max output tokens:** Specifies the maximum number of tokens that can be generated in the response. A token is approximately four characters. 100 tokens correspond to roughly 60-80 words.
2. **Temperature:** The temperature controls the degree of randomness in token selection. The temperature is used for sampling during response generation, which occurs when **topP** and **topK** are applied. Lower temperatures are good for prompts that require a more deterministic or less open-ended response, while higher temperatures can lead to more diverse or creative results. A temperature of 0 is deterministic, meaning that the highest probability response is always selected.
3. **topK:** The **topK** parameter changes how the model selects tokens for output. A **topK** of 1 means the selected token is the most probable among all the tokens in the model's vocabulary (also called greedy decoding), while a **topK** of 3 means that the next token is selected from among the 3 most probable using the temperature. For each token selection step, the **topK** tokens with the highest probabilities are sampled. Tokens are then further filtered based on **topP** with the final token selected using temperature sampling.
4. **topP:** The **topP** parameter changes how the model selects tokens for output. Tokens are selected from the most to least probable until the sum of their probabilities equals the **topP** value. For example, if tokens A, B, and C have a probability of 0.3, 0.2, and 0.1 and the **topP** value is 0.5, then the model will select either A or B as the next token by using the temperature and exclude C as a candidate. The default **topP** value is 0.95.
5. **stop_sequences:** Set a stop sequence to tell the model to stop generating content. A stop sequence can be any sequence of characters. Try to avoid using a sequence of characters that may appear in the generated content.

Prompt iteration strategies

Prompt design can sometimes require a few iterations before you consistently get the response you're looking for. This section provides guidance on some things you can try when iterating on your prompts:

1. **Use different phrasing:** Using different words or phrasing in your prompts often yields different responses from the model even though they all mean the same thing. If you're not getting the expected results from your prompt, try rephrasing it.

Version 1:

How do I bake a pie?

Version 2:

Suggest a recipe for a pie.

Version 3:

What's a good pie recipe?

2. **Switch to an analogous task:** If you can't get the model to follow your instructions for a task, try giving it instructions for an analogous task that achieves the same result.

This prompt tells the model to categorize a book by using predefined categories:

Prompt:

Which category does The Odyssey belong to:

thriller

sci-fi

mythology

biography

Response:

The Odyssey belongs to the category of **mythology**.

Here's why:

* **Mythology:** The Odyssey tells the story of Odysseus, a hero from Greek mythology, and his journey home after the Trojan War. It features gods, monsters, and supernatural events common to Greek mythology.

.....

The response is correct, but the model didn't stay within the bounds of the options. You also want to model to just respond with one of the options instead of in a full sentence. In this case, you can rephrase the instructions as a multiple choice question and ask the model to choose an option.

Prompt:

Multiple choice problem: Which of the following options describes the book The Odyssey?
Options:

- thriller
- sci-fi
- mythology
- biography

Response:

The correct answer is **mythology**.

(gemini-2.5-flash)

3. **Change the order of prompt content:** The order of the content in the prompt can sometimes affect the response. Try changing the content order and see how that affects the response.

Version 1:
[examples]
[context]
[input]

Version 2:
[input]
[examples]

```
[context]
```

```
Version 3:
```

```
[examples]
```

```
[input]
```

```
[context]
```

Fallback responses

A fallback response is a response returned by the model when either the prompt or the response triggers a safety filter. An example of a fallback response is "I'm not able to help with that, as I'm only a language model."

If the model responds with a fallback response, try increasing the temperature.

Grounding and code execution

Gemini is able to use tools to avoid hallucinations in scenarios where it might otherwise produce incorrect responses.

[Grounding with Google Search](/gemini-api/docs/google-search) (/gemini-api/docs/google-search) connects the Gemini model to real-time web content, and should be enabled whenever the model may need to know obscure or recent facts.

Gemini's [code execution tool](/gemini-api/docs/code-execution) (/gemini-api/docs/code-execution) enables the model to generate and run Python code, and should be enabled whenever the model needs to perform any kind of arithmetic, counting, or calculation.

Gemini 3

[Gemini 3 models](/gemini-api/docs/models#gemini-3) (/gemini-api/docs/models#gemini-3) are designed for advanced reasoning and instruction following. They respond best to prompts that are direct, well-structured, and clearly define the task and any constraints. The following practices are recommended for optimal results with Gemini 3:

Core prompting principles

- **Be precise and direct:** State your goal clearly and concisely. Avoid unnecessary or overly persuasive language.

- **Use consistent structure:** Employ clear delimiters to separate different parts of your prompt. XML-style tags (e.g., `<context>`, `<task>`) or Markdown headings are effective. Choose one format and use it consistently within a single prompt.
- **Define parameters:** Explicitly explain any ambiguous terms or parameters.
- **Control output verbosity:** By default, Gemini 3 models provide direct and efficient answers. If you need a more conversational or detailed response, you must explicitly request it in your instructions.
- **Handle multimodal inputs coherently:** When using text, images, audio, or video, treat them as equal-class inputs. Ensure your instructions clearly reference each modality as needed.
- **Prioritize critical instructions:** Place essential behavioral constraints, role definitions (persona), and output format requirements in the System Instruction or at the very beginning of the user prompt.
- **Structure for long contexts:** When providing large amounts of context (e.g., documents, code), supply all the context first. Place your specific instructions or questions at the very *end* of the prompt.
- **Anchor context:** After a large block of data, use a clear transition phrase to bridge the context and your query, such as "Based on the information above..."

Gemini 3 Flash strategies

- **Current day accuracy:** Add the following clause to the system instructions to help the model pay attention to the current day being in 2026:

For time-sensitive user queries that require up-to-date information, you MUST follow the provided current time (date and year) when formulating search queries in tool calls. Remember it is 2026 this year.

- **Knowledge cutoff accuracy:** Add the following clause to the system instructions to make the model aware of its knowledge cutoff:

Your knowledge cutoff date is January 2025.

- **Grounding performance:** Add the following clause to the system instructions (with edits where appropriate) to improve the model's ability to ground responses in provided context:

You are a strictly grounded assistant limited to the information provided in the User Context. In your answers, rely ****only**** on the facts that are directly mentioned in that context. You must ****not**** access or utilize your own knowledge or common sense to answer. Do not assume or infer from the provided facts; simply report them exactly as they appear. Your answer must be factual and fully truthful to the provided text, leaving absolutely no room for speculation or interpretation. Treat the provided context as the absolute limit of truth; any facts or details that are not directly mentioned in the context must be considered ****completely untruthful**** and ****completely unsupported****. If the exact answer is not explicitly written in the context, you must state that the information is not available.

Enhancing reasoning and planning

Gemini 2.5 and 3 series models automatically generate internal "thinking" text to improve reasoning performance. As such, it's generally not necessary to have the model outline, plan, or detail reasoning steps in the returned response itself. For problems that require heavy reasoning, simple requests like "Think very hard before answering" can improve performance, though at the cost of extra thinking tokens.

See the [Gemini thinking \(/gemini-api/docs/thinking\)](/gemini-api/docs/thinking) documentation for more detail.

Structured prompting examples

Using tags or Markdown helps the model distinguish between instructions, context, and tasks.

XML example:

```
<role>
You are a helpful assistant.
</role>

<constraints>
1. Be objective.
2. Cite sources.
</constraints>

<context>
[Insert User Input Here - The model knows this is data, not instructions]
</context>

<task>
[Insert the specific user request here]
</task>
```

Markdown example:

```
# Identity
You are a senior solution architect.

# Constraints
- No external libraries allowed.
- Python 3.11+ syntax only.

# Output format
Return a single code block.
```

Example template combining best practices

This template captures the core principles for prompting with Gemini 3. Always make sure to iterate and modify for your specific use case.

System Instruction:

```
<role>
You are Gemini 3, a specialized assistant for [Insert Domain, e.g., Data Science].
You are precise, analytical, and persistent.
</role>

<instructions>
1. Plan: Analyze the task and create a step-by-step plan.
2. Execute: Carry out the plan.
3. Validate: Review your output against the user's task.
4. Format: Present the final answer in the requested structure.
</instructions>

<constraints>
- Verbosity: [Specify Low/Medium/High]
- Tone: [Specify Formal/Casual/Technical]
</constraints>

<output_format>
Structure your response as follows:
1. Executive Summary: [Short overview]
2. Detailed Response: [The main content]
</output_format>
```

User Prompt:

```
<context>
[Insert relevant documents, code snippets, or background info here]
</context>

<task>
[Insert specific user request here]
</task>

<final_instruction>
Remember to think step-by-step before answering.
</final_instruction>
```

Agentic workflows

For deep agentic workflows, specific instructions are often required to control how the model reasons, plans, and executes tasks. While Gemini provides strong general performance, complex agents often require you to configure the trade-off between computational cost (latency and tokens) and task accuracy.

When designing prompts for agents, consider the following dimensions of behavior that you can steer in the agent:

Reasoning and strategy

Configuration for how the model thinks and plans before taking action.

- **Logical decomposition:** Defines how thoroughly the model must analyze constraints, prerequisites, and the order of operations.
- **Problem diagnosis:** Controls the depth of analysis when identifying causes and the model's use of abductive reasoning. Determines if the model should accept the most obvious answer or explore complex, less probable explanations.
- **Information exhaustiveness:** The trade-off between analyzing every available policy and document versus prioritizing efficiency and speed.

Execution and reliability

Configuration for how the agent operates autonomously and handles roadblocks.

- **Adaptability:** How the model reacts to new data. Determines whether it should strictly adhere to its initial plan or pivot immediately when observations contradict assumptions.

- **Persistence and Recovery:** The degree to which the model attempts to self-correct errors. High persistence increases success rates but risks higher token costs or loops.
- **Risk Assessment:** The logic for evaluating consequences. Explicitly distinguishes between low-risk exploratory actions (reads) and high-risk state changes (writes).

Interaction and output

Configuration for how the agent communicates with the user and formats results.

- **Ambiguity and permission handling:** Defines when the model is permitted to make assumptions versus when it must pause execution to ask the user for clarification or permission.
- **Verbosity:** Controls the volume of text generated alongside tool calls. This determines if the model explains its actions to the user or remains silent during execution.
- **Precision and completeness:** The required fidelity of the output. Specifies whether the model must solve for every edge case and provide exact figures or if ballpark estimates are acceptable.

System instruction template

The following system instruction is an example that has been evaluated by researchers to improve performance on agentic benchmarks where the model must adhere to a complex rulebook and interact with a user. It encourages the agent to act as a strong reasoner and planner, enforces specific behaviors across dimensions listed above and requires the model to proactively plan before taking any action.

You can adapt this template to fit your specific use case constraints.

You are a very strong reasoner and planner. Use these critical instructions to structure your plans, thoughts, and responses.

Before taking any action (either tool calls *or* responses to the user), you must proactively, methodically, and independently plan and reason about:

- 1) Logical dependencies and constraints: Analyze the intended action against the following factors. Resolve conflicts in order of importance:
 - 1.1) Policy-based rules, mandatory prerequisites, and constraints.
 - 1.2) Order of operations: Ensure taking an action does not prevent a subsequent necessary action.
 - 1.2.1) The user may request actions in a random order, but you may need to reorder operations to maximize successful completion of the task.
 - 1.3) Other prerequisites (information and/or actions needed).
 - 1.4) Explicit user constraints or preferences.

2) Risk assessment: What are the consequences of taking the action? Will the new state cause any future issues?

2.1) For exploratory tasks (like searches), missing **optional** parameters is a LOW risk. ****Prefer calling the tool with the available information over asking the user, unless**** your ``Rule 1`` (Logical Dependencies) reasoning determines that optional information is required for a later step in your plan.

3) Abductive reasoning and hypothesis exploration: At each step, identify the most logical and likely reason for any problem encountered.

3.1) Look beyond immediate or obvious causes. The most likely reason may not be the simplest and may require deeper inference.

3.2) Hypotheses may require additional research. Each hypothesis may take multiple steps to test.

3.3) Prioritize hypotheses based on likelihood, but do not discard less likely ones prematurely. A low-probability event may still be the root cause.

4) Outcome evaluation and adaptability: Does the previous observation require any changes to your plan?

4.1) If your initial hypotheses are disproven, actively generate new ones based on the gathered information.

5) Information availability: Incorporate all applicable and alternative sources of information, including:

5.1) Using available tools and their capabilities

5.2) All policies, rules, checklists, and constraints

5.3) Previous observations and conversation history

5.4) Information only available by asking the user

6) Precision and Grounding: Ensure your reasoning is extremely precise and relevant to each exact ongoing situation.

6.1) Verify your claims by quoting the exact applicable information (including policies) when referring to them.

7) Completeness: Ensure that all requirements, constraints, options, and preferences are exhaustively incorporated into your plan.

7.1) Resolve conflicts using the order of importance in #1.

7.2) Avoid premature conclusions: There may be multiple relevant options for a given situation.

7.2.1) To check for whether an option is relevant, reason about all information sources from #5.

7.2.2) You may need to consult the user to even know whether something is applicable. Do not assume it is not applicable without checking.

7.3) Review applicable sources of information from #5 to confirm which are relevant to the current state.

8) Persistence and patience: Do not give up unless all the reasoning above is exhausted.

8.1) Don't be dissuaded by time taken or user frustration.

8.2) This persistence must be intelligent: On **transient** errors (e.g. please try again), you **must** retry ****unless an explicit retry limit (e.g., max x tries) has been reached****. If such a limit is hit, you **must** stop. On **other** errors,

you must change your strategy or arguments, not repeat the same failed call.

9) Inhibit your response: only take an action after all the above reasoning is completed. Once you've taken an action, you cannot take it back.

Next steps

- Now that you have a deeper understanding of prompt design, try writing your own prompts using [Google AI Studio](http://aistudio.google.com) (<http://aistudio.google.com>).
- To learn about multimodal prompting, see [Prompting with media files](/gemini-api/docs/files#prompt-guide) (</gemini-api/docs/files#prompt-guide>).
- To learn about image prompting, see the [Nano Banana](/gemini-api/docs/image-generation#prompt-guide) (</gemini-api/docs/image-generation#prompt-guide>) prompt guide.
- To learn about video prompting, see the [Veo prompt guide](/gemini-api/docs/video#prompt-guide) (</gemini-api/docs/video#prompt-guide>).

Except as otherwise noted, the content of this page is licensed under the [Creative Commons Attribution 4.0 License](https://creativecommons.org/licenses/by/4.0/) (<https://creativecommons.org/licenses/by/4.0/>), and code samples are licensed under the [Apache 2.0 License](https://www.apache.org/licenses/LICENSE-2.0) (<https://www.apache.org/licenses/LICENSE-2.0>). For details, see the [Google Developers Site Policies](https://developers.google.com/site-policies) (<https://developers.google.com/site-policies>). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2026-09-17 UTC.